



RELATÓRIO FINAL: ANÁLISE AVANÇADA DE CONGESTIONAMENTO TCP USANDO EBPF

1. Introdução

O presente relatório documenta o desenvolvimento e a aplicação da ferramenta *TCP Congestion Observatory (TCP-CO)*, baseada na tecnologia eBPF (*Extended Berkeley Packet Filter*). O objetivo principal consistiu em monitorar, em tempo real, o comportamento das conexões TCP no kernel do Linux, extraindo métricas críticas de controle de congestionamento sem a necessidade de modificação do código-fonte do sistema operativo. A análise avaliou o comportamento dos algoritmos TCP CUBIC e TCP RENO perante cenários de rede ideais, com perda de pacotes e com alta latência.

2. Metodologia e Instrumentação

A coleta de dados foi realizada através da instrumentação de *hooks* do kernel (`tcp_ack`) utilizando programas eBPF escritos em C, exportados para o *user-space* via uma API em Python e visualizados através do *Jupyter Notebook* (Pandas e Matplotlib). Os experimentos foram conduzidos na interface de *loopback* (`lo`), utilizando a ferramenta `iperf3` para geração de tráfego constante. Condições anômalas foram emuladas via `tc netem`.

Nota Metodológica (Conversão de RTT): A latência capturada pelo eBPF é extraída do kernel como *Smoothed Round Trip Time* em microssegundos (`srtt_us`). Para rigor analítico, o *script* de visualização foi configurado para aplicar a conversão matemática $srtt_us / 1000.0$, apresentando todos os gráficos e estatísticas descritivas em milissegundos (**RTT em ms**).

3. Síntese dos Experimentos

- **Experimento 1 (Rede Padrão):** Num ambiente ideal de loopback (latência < 0.1 ms), o CUBIC estabilizou a sua janela média em 62 pacotes, focando no controle. O RENO, utilizando sua política agressiva, saturou o link virtual, alcançando médias superiores a 220 pacotes e picos de 249 pacotes.
- **Experimento 2 (1% de Perda):** A inserção de falhas provou a diferença nos mecanismos de redução. O RENO foi severamente estrangulado (caindo para uma média de míseros 5.77 pacotes) devido aos sucessivos cortes de 50% na janela. O CUBIC, com o seu fator de redução multiplicativa de ~30%, conseguiu sustentar uma média superior (8 pacotes), mas ambos sofreram milhares de retransmissões.
- **Experimento 3 (RTT Elevado):** Comprovou-se a correlação inversa: quanto maior a latência (100ms, 200ms, 500ms), mais lenta é a fase de *Slow Start* (devido à demora dos ACKs) e menor tende a ser a janela de congestionamento estabilizada, com o objetivo de respeitar o limite do *buffer* da rede.
- **Experimento 4 (Comparativo CUBIC vs RENO):** Utilizando médias móveis para filtrar o ruído, confirmou-se visualmente que o CUBIC mantém a estabilidade da curva (teto de 133 pacotes), enquanto o RENO exibe forte instabilidade em "dente de serra" (teto de 249 pacotes), embora ambos tenham sofrido o mesmo volume absoluto de retransmissões perante o stress.

3.1. Bônus: Máquina de Estados e Alertas

A variável `icsk_ca_state` foi monitorada e traduzida via Python, plotando um gráfico de degraus que comprovou a transição exata do estado `0 - Open` para `3 - Recovery` durante as perdas.

Adicionalmente, implementou-se um *script* de alerta automático que varre o DataFrame e emite alertas visuais sempre que detecta a condição exigida (queda de `Cwnd` $\geq 50\%$ simultânea ao aumento de retransmissões), comprovando o pleno funcionamento do observatório.

4. Análise Crítica (Respostas às Questões)

1. Como a janela de congestionamento evolui durante o Slow Start? Durante o *Slow Start* (Partida Lenta), a janela de congestionamento (`Cwnd`) inicia-se em um valor baixo (padrão de 10 pacotes no Linux) e cresce de maneira exponencial. A cada confirmação de recebimento (ACK) válida, a janela é incrementada, o que resulta na duplicação do tamanho da janela a cada ciclo completo de RTT, permitindo que o algoritmo descubra a capacidade do enlace rapidamente.

2. Em que momento ocorre a transição para Congestion Avoidance? Como isso foi identificado? A transição ocorre no momento em que a `Cwnd` atinge o valor limiar definido pela variável `sssthresh` (Slow Start Threshold), ou antecipadamente, caso ocorra perda de pacote. Nos gráficos gerados, esta transição foi visualmente identificada no ponto de inflexão da curva no início da conexão: a linha deixa de subir quase verticalmente (fase exponencial) e passa a exibir uma inclinação muito mais suave, estabilizando num platô (CUBIC) ou crescendo de forma aditiva linear (RENO).

3. Qual o impacto da perda de pacotes sobre cwnd, sssthresh e throughput? A perda atua como o principal gatilho de congestionamento. Ao ser detectada, a `Cwnd` sofre uma drástica redução (corte rigoroso de 50% no RENO; corte para ~70% no CUBIC). Imediatamente, o `sssthresh` é rebaixado dinamicamente para se igualar a esse novo valor da janela. O *throughput* sofre uma queda severa, pois o remetente injeta menos dados na rede e gasta capacidade operacional processando retransmissões (que ultrapassaram 14.000 pacotes nos nossos testes).

4. Qual algoritmo apresentou melhor desempenho? Justifique. O CUBIC demonstrou melhor desempenho global, justificando sua posição como padrão no Linux. Embora o RENO tenha apresentado médias de `Cwnd` superiores num ambiente irreal (localhost sem perdas), o Experimento 2 provou a sua fragilidade: perante apenas 1% de perda, o RENO colapsou. O CUBIC, através da sua função de crescimento cúbico, recupera a janela rapidamente após eventos de perda e mantém maior estabilidade, evitando os picos agressivos que causariam saturação nas redes.

5. O RTT influenciou o desempenho dos algoritmos? Como? Sim, de forma determinante. Conforme provado no Experimento 3, o aumento do tempo de viagem do pacote (RTT para 100ms, 200ms e 500ms) dilata o tempo necessário para o crescimento da `Cwnd`. Como a injeção de novos pacotes depende da chegada dos ACKs, um RTT elevado obriga a janela a crescer num ritmo muito mais lento e estabilizar em patamares menores, limitando diretamente o *Throughput* máximo da aplicação.

6. Quais limitações foram encontradas no uso do eBPF para observabilidade TCP? A principal limitação recai na leitura em altíssima velocidade em ambientes de latência microscópica (como o *loopback*). Como a entrega ocorre em frações de microssegundo, capturar a fila exata de pacotes "em voo" (`packets_out`) exigiu filtragem cuidadosa, pois o kernel esvazia a fila quase instantaneamente. Além disso, o eBPF exige privilégios de `root` contínuos e o manuseio complexo de conversões de *structs* em C para o *user-space* em Python.

7. Quais vantagens o eBPF apresenta em relação à modificação direta do kernel? A segurança e o dinamismo. Inserir código diretamente no kernel exige recompilação do Sistema Operativo e qualquer erro pode causar um travamento fatal (*Kernel Panic*). O eBPF roda numa Máquina Virtual isolada (Sandbox) que verifica a segurança do código previamente. Além disso, os programas podem ser ativados (`autoattach`) e removidos em tempo real, sem necessidade de reiniciar os serviços de rede ou a máquina.

8. Quais seriam aplicações práticas dessa ferramenta em ambientes reais? Em produção, o TCP-CO poderia ser utilizado para:

- **Monitoramento de Data Centers e CDNs:** Detectar clientes sofrendo com *bufferbloat* ou estrangulamento de banda em rotas internacionais.
- **Auditoria de Infraestrutura:** Identificar roteadores defeituosos que descartam pacotes silenciosamente, rastreando disparos anômalos da variável de retransmissão e alterações bruscas no `ca_state`.
- **Otimização de Servidores:** Auxiliar a engenharia de tráfego na validação empírica de qual algoritmo (CUBIC, BBR) fornece a melhor taxa de transferência para o perfil específico dos usuários do serviço.